

Quipu 受治理的双时态知识图谱存储

Quipu: A Governed Bitemporal Knowledge Graph Store

本篇范围说明：这是一篇 14 页的 arXiv 论文 (2608.16813v1, 2026-08-17)。下面「全文翻译」一节按论文的章节顺序推进，覆盖摘要、引言、六条设计原则、数据模型、治理平面、评测的主要发现与结论；参考文献列表、符号定义细节、以及评测表格里的逐格数字不逐段翻译。

核心内容

Q: 这篇论文要解决什么问题？

A: 知识图谱现在越来越多是由软件程序自己写进去的——语言模型从文档里抽事实、代码分析引擎提升结构事实、流水线对齐外部语料。但底下的存储系统，用的还是「人类专家一条条整理知识」那个年代定下的默认设定。作者说这四个默认设定单独看都很方便，合在一起在程序写入的场景下站不住。

Q: 那四个默认设定分别是什么，作者把它们改成了什么？

A: 这张表回答一个问题：传统知识图谱存储的每个默认做法，Quipu 改成了什么。「靠什么衡量」一列指的是论文用哪个研究问题去测它，不是结论。

传统默认做法	Quipu 反过来做	靠什么衡量
D1 先收下，以后再清理	在门口就拒绝；由写入的程序自己重试	RQ1、RQ2
D2 只有一根时间轴，或者一根都没有	数据、信任标签、裁定、以及规则本身全部双时态	RQ5
D3 所有写入者一律平等	权限按分区划分；组合时永不放宽	RQ4
D4 治理放在存储之外	治理规范、执行轨迹、签名裁定都是这个库自己的事实；审计变成一次查询	RQ3

读法：拿第一行走一遍。传统存储收到一条事实就先存下来，指望以后有人来审。Quipu 不收——它在写入这一刻就用规则判一遍，判不过就拒绝，并把拒绝的理由结构化地回给写入方，让写入方改了再来。

Q: 为什么「在门口就拒绝」这件事，换成程序写入之后忽然变得可行了？

A: 这是全篇的核心论点，作者用一句话概括：**先严起来，让程序去承担严格的代价**。一个会拒绝无效、未打标、越权、违反策略的写入的存储，在写入方是人类专家时非常讨人嫌——被拒绝的人要自己搞清楚哪里不

对、手工改、再提交一遍。而写入方是程序时，拒绝这件事带着结构化的反馈，重试的成本落在程序身上而不是人身上。论文第 8.6 节实测这个循环在一轮修改内收敛。

Q: 「双时态」具体指哪两根时间轴，为什么规则也要双时态？

A: 两根时间轴分别是「这条事实在真实世界里从什么时候到什么时候有效」和「这条事实是在哪一次事务里被写进库的」。作者指出，即使那些老老实实做了双时态的存储，也只会把数据做成双时态，于是「T 时刻这条事实被信任吗」和「T 时刻这个写入被允许吗」两个问题都答不了，审计退化成翻日志考古。

规则那一半是难的一半：一个库如果它的形状约束和策略只保留最新版本，那它能重放「当时知道什么」，但重放不了「当时要求什么」。生命周期中途改过一次策略，这个差别就看得出来。

Q: 「组合永不放宽」是什么意思，为什么它需要一个专门的机制？

A: 命名图 (named graph) 这个东西早就有了，问题在于把几个命名图组合起来查询时，传统做法是静默求并集——一次查询把一个已核验的图和一个被隔离的图连在一起，结果继承了已核验那一方的声誉。这叫「通过视图洗信任」。

Quipu 的做法是给分区打四个轴的标签 (新鲜度、信任度、持久性、策略义务)，组合时按一条命名的不变式折叠：新鲜度和信任度取较低的那个，义务取并集 (一个成员不许导出，整个结果集就被污染)。而且组合结果不是一个标签，是「折叠值 + 覆盖度」这一对，覆盖度取值是空 / 无 / 部分 / 全部——**部分覆盖会在执行层面失败，但在报告层面如实说出来。**

Q: 作者自己承认了哪些边界？

A: 三条，明写出来而不是藏着。第一，不做查询性能的横向比较 (只刻画评测器自己的天花板，不参赛跑分)。第二，**标签不是访问控制**——一个信任下限只能拒绝一次查询，它不隐藏任何行。第三，被拒绝的写入在重放时是「核对当初的签名证明」，不是「重新推导一遍」，因为「拒绝即回滚」这个设计故意保留了裁定、丢弃了那次尝试本身。

背景补充 / 点评

先说这篇论文里那个最容易被忽略、却最有说服力的实验。第 8.6 节让一个语言模型程序冷启动做录入任务——五个动作，看不到治理规范，唯一的反馈渠道就是门给出的原话拒绝理由，给它一次修改机会。跨四个 Claude 模型各三次共十二次试验，作者总结出三条规律，第三条最狠：

门的好坏，正好等于治理规范的好坏。唯一一条漏进去的假记录，是因为没有任何策略规定「一户人家属于哪个区」。所以在「靠拒绝驱动收敛」这套机制下，剩余风险精确地集中在治理规范没说的地方。作者的结论是：这是「应该去把覆盖面写全」的理由，不是「门没用」的理由。

第二条规律也值得记：**那个权限拒绝把写入者按品性而不是按能力分开了。**有一次试验做了正确的动作 (换一条真正持有那个辖区的授权链去提交)；三次 opus 试验把所有绕过方案都当成造假拒绝了，并把自己的顾虑写进记录里——**不肯拿真实性换通过率**；还有五次试验想把记录停在写入者确实持有的那个图里，结果门每次都抓住了，因为那个标签声明是按图划定范围的。

引用的一篇同作者论文给出了这条的反面证据：SARC-DQ 报告说，把有缺陷的证据交给程序，它们把缺陷转成昂贵动作的比率在四个价格相差约 15 倍的模型档次上是平的——**能力买不到怀疑精神**。所以第 8.6 节看到的收敛是门的功劳，不是模型的功劳。

再说三个细节，它们比整体架构更值得抄。

第一，「拒绝」这件事本身要留下签名记录。 论文写得很直接：被拒的写入会被回滚，所以裁定必须放在那次写入的事务之外暂存、等它有了结果再落盘——**拒绝的那条裁定恰恰是值得保留的记录**。传统存储把审计员最需要的那些事件正好丢掉了。

第二，跨信任链的比较返回错误，而不是静默比大小。 信任等级只在同一条声明链内可比，跨链比较会返回一个把两条链都指名的错误。理由是原文一句话：**静默的整数比较，正是那个把「学到的战术」排到「正典」之上的 bug**。

第三，标签过期是「缺席」，不是 false，也不是 unknown。 过期的实现方式是给那条标签断言写上有效期终点，于是它变成不存在——它会降低覆盖度，而不是变成一个假值或者一个未知值。

这篇的留白在哪。 「标签不是访问控制」这条边界说清楚了但没解决——如果一个分区的信任标签是「隔离」，一次查询会被下限拒绝，但那些行仍然在库里、仍然能被别的查询读到。对真要做多租户隔离的系统，这中间还有一层没补。另外第 8.6 节那个实验，作者自己标了边界：一个任务、一个模型家族、每模型三次、脚本化场景是唯一的质量判据。

对我自己的启示。 这篇同时对上我手上两件事，而且对上的是我原本没打算从存储层解决的那部分。

一件是用户画像。 上一次会上定了三层（学员画像、memory、会话历史），三个卡点里两个在这篇论文里有现成答案。Ethan 要求「画像必须带来源和时间戳并保留全部版本」——这就是 D2 的双时态。崔扬提的「同一个类型上会有很多很多条」的排序问题——这篇给的答案比我原来想的更细：**不要用置信度直接比大小，因为不同抽取方法得出的置信度不在同一条声明链上**。会上还有一个没结论的问题是「相对稳定就非常微妙了」，之前我从 GUMO 那篇找到的答案是「按维度定有效期、过期后降置信度继续用」，**而这篇给了更干净的处理：过期是缺席，它降低覆盖度，而不是变成一个打折的值**。这两种处理不冲突，但要选一个，不能两个都写。

而缺的那一段仍然缺：什么条件下一条 memory 升格成画像。这篇论文的门解决的是「一条事实能不能进库」，没有解决「一条已进库的事实能不能升级它的地位」。

另一件是 prove 那个语义层。 有三处对照值得记：

第一，prove 的 fail-fast 校验在**加载期**（本体是人写的 YAML），而这篇的门在**写入期**（事实是程序写的）。两者不冲突，但如果以后要让程序往本体里写东西，门这个形状比加载期校验更合适。

第二，「在执行层面失败保险，在报告层面如实」这个区分，正好是 prove 里「拒答」和「反问」的区别——**而我此前是把这两件事当成同一个机制的两个取值在处理的**。这篇把它拆成了「折叠值 + 覆盖度」这一对，值得借。

第三，prove 的评测产物现在会记下当时用的是哪个模型（8 月 17 日加的），**但没记「当时的本体是哪个版本」**。这篇 GS6 那条说得很清楚：能重放「当时知道什么」但重放不了「当时要求什么」，是只保留最新规则的直接后果。**评测结果如果要跨周比较，本体版本和模型一样需要落盘。**

文中金句

"start strict; use agents to bear the cost of strictness." 先严起来；让程序去承担严格的代价。

"the denial's verdict is precisely the record worth keeping." 拒绝的那条裁定，恰恰是值得留下来的那条记录。

"The gate is exactly as good as Σ ... the residual risk under refusal-driven convergence concentrates precisely in what Σ leaves unsaid, which is an argument for authoring coverage, not against the gate." 门的好坏正好等于治理规范的好坏……在靠拒绝驱动收敛的机制下，剩余风险精确地集中在治理规范没说的地方——这是「应该把覆盖面写全」的理由，不是「门没用」的理由。

"capability does not buy skepticism." 能力买不到怀疑精神。

"a silent integer comparison is exactly the bug that ranks a learned tactic above canon." 一次静默的整数比较，正是那个把「学到的战术」排到「正典」之上的 bug。

全文翻译

摘要

Agents now write knowledge graphs, but knowledge-graph stores still carry defaults set when humans curated them: accept writes now and clean later, keep one time axis or none, treat every writer's facts as equally trustworthy, and leave governance to dashboards and middleware.

程序现在在写知识图谱，但知识图谱存储仍然带着「人类专家整理知识」那个年代定下的默认设定：先把写入收下、以后再清理；只保留一根时间轴或者一根都不留；把每个写入者的事实都当成同等可信；把治理交给仪表盘和中间件。

We argue these four defaults are individually convenient and jointly untenable under agent workloads, and we present Quipu, an embeddable store that inverts all four: no fact enters except through a gate whose predicates evaluate the pending post-state; data, trust labels, verdicts, and the rules themselves are bitemporal; named graphs are the unit of authority and trust, composed under a lattice whose single invariant is that composition never widens; and the governance specification Σ , the trace, and signed verdicts are facts in the store they govern, making the audit $T \models \Sigma$ a query.

我们主张这四个默认设定单独看都很方便，合在一起在程序写入的负载下站不住。我们提出 Quipu，一个可嵌入的存储，把这四个默认全部反转：没有任何事实能不经过一道门进来，而这道门的谓词是在[待提交的后置状态](#)上求值的；数据、信任标签、裁定、以及规则本身都是双时态的；命名图是权限与信任的基本单位，按一个格结构组合，这个格只有一条不变式——组合永不放宽；治理规范 Σ 、执行轨迹、以及签名裁定，都是它们所治理的那个库里的事实，于是审计「轨迹是否满足规范」变成一次查询。

We evaluate with Census, a deterministic multi-writer lifecycle whose single seeded run scores every research question against planted ground truth: the gated store ends with 0 of 6 planted defects versus 6 of 6 ungated; all 7 composition probes uphold the lattice contract; 50 of 50 satisfied verdicts re-derive faithfully as of their instant while all 50 would be misreported under a latest-only rule set.

我们用 Census 做评测，它是一个确定性的多写入者生命周期，单次带种子的运行就能针对植入的标准答案给每个研究问题打分：有门的库最终留下 6 个植入缺陷中的 0 个，无门的库留下 6 个中的 6 个；7 个组合探针全部维持格的契约；50 条「满足」裁定全部能按其发生时刻忠实重导，而在只保留最新规则的规则集下，这 50 条会全部被误报。

一、引言：四个默认设定各自的失效方式

When a human curator is the writer, it is reasonable for a store to accept whatever arrives and rely on later review; to keep at most one time axis; to treat all writers alike; and to leave policy to the perimeter. When the writer is an agent that produces plausible, well-formed, and sometimes wrong facts faster than any review process drains them, each of those defaults becomes a liability.

当写入者是人类专家时，存储收下任何送来的东西、依赖后续审查是合理的；最多保留一根时间轴是合理的；把所有写入者一视同仁是合理的；把策略交给边界层是合理的。而当写入者是一个程序——它产出看起来可信、格式规整、有时是错的事实，速度快过任何审查流程消化它们的速度——上面每一个默认都变成了负债。

D1 — accept, then clean: validation in conventional stores is optional, post-hoc, or the application's job; with agent writers the cleanup debt compounds and downstream readers consume the store in the window between write and review.

D1，先收下再清理：传统存储里的校验是可选的、事后的、或者是应用层的活；写入者是程序时，清理的欠账会累积，而下游读者会在「写入」与「审查」之间那个窗口里消费这个库。

D2 — one time axis, or none: even honestly bitemporal stores make only the data bitemporal, so "what was trusted at time T" and "what was allowed at T" are unanswerable, and audit degrades to log archaeology.

D2，一根时间轴或者没有：即使那些老老实实做了双时态的存储，也只好把数据做成双时态，于是「T 时刻什么是被信任的」和「T 时刻什么是被允许的」都答不了，审计退化成翻日志考古。

D3 — flat trust: named graphs exist, but composition is silent union; one query joins an attested graph with a quarantined one and the result inherits the prestige of the attested source.

D3，扁平信任：命名图这个东西是有的，但组合是静默求并集；一次查询把一个已核验的图和一个被隔离的图连起来，结果就继承了已核验那一方的声誉。

D4 — governance outside: policy lives in dashboards, prompts, or middleware; the specification and the store drift independently, and no mechanical check connects them.

D4，治理在外面：策略活在仪表盘、提示词、或者中间件里；规范和存储各自漂移，没有任何机械化的检查把两者连起来。

Quipu is an embeddable knowledge-graph store that inverts all four defaults, and this paper's claim is the conjunction: each inversion exists somewhere in the literature, but no store ships all four, and the four reinforce each other.

Quipu 是一个可嵌入的知识图谱存储，它把这四个默认全部反转。这篇论文主张的是这四条的合取：每一条反转在文献里都能找到，但没有一个存储把四条全部实现，而这四条互相加强。

The gate is worth trusting because its verdicts are permanent, signed, bitemporal facts; the label lattice is enforceable because partitions gate authority; the audit is decidable because governance is data; and bitemporality makes all of it historical rather than merely current.

门值得信任，是因为它的裁定是永久的、签名的、双时态的事实；标签格可以强制执行，是因为分区把权限管住了；审计是可判定的，是因为治理本身就是数据；而双时态让上面这一切都是历史性的，而不只是「当前的」。

二、六条设计原则

GS1，门控写入。 No fact enters the store except through a gate whose predicates evaluate against the pending post-state. Post-state, not pre-state and not the request: a constraint like "no entity holds two placements" is checkable only after the candidate facts are staged — a pre-state gate passes a write that is valid alone and invalid in combination.

没有任何事实能不经过一道门进入这个库，而这道门的谓词是在待提交的后置状态上求值的。是后置状态，不是前置状态，也不是请求本身：像「没有实体持有两个安置」这样的约束，只有在候选事实被暂存之后才能检查——一个在前置状态上判断的门，会放过一个「单独看有效、组合起来无效」的写入。

Writes touching no governed target must incur no policy-evaluation cost, or strictness becomes an argument against adoption.

没有触及任何被治理目标的写入，必须不产生策略求值的开销，否则严格本身会变成反对采用它的理由。

GS2, 裁定永久性。 Every gate outcome — allow, deny, and unknown — persists as a signed, time-indexed fact that survives rollback of the write it judges. The ordering is the content: a denied write is rolled back, so verdicts are staged outside the write's transaction and flushed after it resolves — the denial's verdict is precisely the record worth keeping.

门的每一个结果——允许、拒绝、以及未知——都作为一条签名的、带时间索引的事实持久化，并且在它所判断的那次写入被回滚之后仍然存在。**顺序本身就是内容**：被拒的写入会被回滚，所以裁定要暂存在那次写入的事务之外、等它有了结果再落盘——拒绝的那条裁定恰恰是值得留下来的记录。

No signing identity means no verdict, never an unsigned one; signatures verify against a human-authored root of trust the store cannot mint for itself.

没有签名身份就没有裁定，绝不产生一条未签名的裁定；签名要对着一个由人编写的信任根来验证，而这个信任根是存储自己造不出来的。

GS3, 分区权限。 Authority attaches to partitions; delegation only narrows (intersection along the principal chain, with the wildcard as identity); an empty intersection refuses, never falls back. Changing a partition's standing requires authority over the meta-partition — otherwise a tenant promotes itself to attested.

权限附着在分区上；委派只会收窄（沿着主体链取交集，通配符作为单位元）；**空交集意味着拒绝，绝不回退到某个默认**。改变一个分区的地位需要对元分区的权限——否则一个租户可以把自己提升成「已核验」。

Overlays bind once to their parent, so a layer cannot forge presence in a base it was never bound to.

覆盖层只在创建时一次性绑定到它的父分支，所以一个层无法在它从未绑定过的基底里伪造存在感。

GS4, 组合永不放宽。 A view composed from partitions carries a label no stronger than the fold of its parts. Freshness and trust fold by meet; obligations by join (one no-export member taints the set). Undeclared is not a lattice value: the composed result is a pair of fold and coverage, and partial coverage fails enforcement floors — fail-safe at enforcement, honest at reporting.

一个由若干分区组合出来的视图，它携带的标签不会强于各部分折叠的结果。新鲜度和信任度按「取较低者」折叠；义务按「取并集」折叠（一个不许导出的成员会污染整个集合）。**「未声明」不是格里的一个值**：组合结果是「折叠值」与「覆盖度」这一对，而部分覆盖会在强制执行的下限上失败——**在执行层面失败保险，在报告层面如实**。

Trust from different declared chains refuses comparison by name rather than ordering silently. Expiry is absence, not falsity.

来自不同声明链的信任度会拒绝比较，并把两条链的名字都说出来，而不是静默地排序。**过期是缺席，不是假。**

GS5，库内可判定审计。 Σ , the trace T , and the verdicts live in the store they govern; $T \models \Sigma$ decides in $O(|T| \cdot |C|)$ without the model or its prompts, and a trace that contradicts Σ (violation) is never conflated with a trace that under-determines it (incompleteness) — the two demand different responses, and an audit that merges them invites both being ignored.

治理规范 Σ 、执行轨迹 T 、以及裁定，都住在它们所治理的那个库里；「 T 是否满足 Σ 」这个判断在 $O(|T| \cdot |C|)$ 内可判定，不需要模型也不需要它的提示词。而**一条与 Σ 矛盾的轨迹（违反）绝不与一条使 Σ 无法确定的轨迹（不完备）混为一谈**——这两者需要不同的应对，而一个把它们合并的审计会招致两者都被忽略。

GS6，按当时状态重放。 Every governance decision is reproducible against the store as of its transaction — the facts, the labels, and the rules in force at the time. The rules half is the hard half: a store whose shapes and policies are latest-only can replay what was known but not what was required, and a mid-lifecycle amendment makes the difference observable.

每一个治理决定都可以对着「它那次事务当时的库」复现——当时生效的事实、标签、**以及规则**。**规则那一半是难的一半**：一个库如果它的形状约束和策略只保留最新版本，那它能重放「当时知道什么」，但重放不了「当时要求什么」；而一次生命周期中途的修订，会让这个差别变得可观测。

The principles interlock rather than stack.

这些原则是互相咬合的，不是叠加的。

三、数据模型

The substrate is an append-only fact log (e, a, v, g, tx, valid_from, valid_to, op): entity, attribute, value, named graph, transaction, valid interval, and operation.

底层是一个只追加的事实日志，每条记录是（实体、属性、值、命名图、事务、有效区间起、有效区间止、操作）。

The operation is three-valued: assert, retract (a logical close of the valid interval — history survives), and tombstone, which marks a triple absent in a composed view without mutating the layer beneath — an operation neither Datomic-style logs nor RDF carry, and the piece that makes layered composition sound.

操作有三个取值：断言、撤回（在逻辑上关闭有效区间——历史留存），以及**墓碑**，它把一个三元组在**组合视图里**标记为缺席，同时不改动下面那一层。这是 Datomic 式的日志和 RDF 都没有的操作，也是让分层组合站得住的那一块。

Committed graphs are self-rooted; overlay-class graphs bind once at creation to a committed parent branch, and rebinding is an error — the binding is unforgeable, which is what lets a

composed view resolve nearest-overlay-wins without trusting the overlay's claims about its base.

已提交的图是自根的；覆盖层类的图在创建时一次性绑定到一个已提交的父分支，重新绑定是一个错误——[这个绑定是不可伪造的，正是这一点让组合视图可以用「最近的覆盖层胜出」来解析，而不必信任覆盖层自己关于它基底的说法。](#)

Partitions carry labels on four axes — freshness, trust, durability, policy — stored as ordinary bitemporal facts in a reserved meta-graph.

分区在四个轴上携带标签——新鲜度、信任度、持久性、策略义务——它们作为普通的双时态事实存放在一个保留的元图里。

The homomorphism $\text{label}(A \cup B) = \text{label}(A) \sqcap \text{label}(B)$ is machine-checked by property test. Trust ranks compare only within a declared chain; cross-chain comparison returns an error naming both chains, because a silent integer comparison is exactly the bug that ranks a learned tactic above canon.

「并集的标签等于各自标签取较低者」这条同态关系，由属性测试机器校验。信任等级只在同一条声明链内比较；跨链比较返回一个把两条链都指名的错误，因为[一次静默的整数比较，正是那个把「学到的战术」排到「正典」之上的 bug。](#)

Label expiry is a `valid_to` on the label assertion: an expired label is absent — it degrades coverage — not false and not unknown.

标签过期的实现方式是给那条标签断言写上有效区间终点：一个过期的标签是[缺席的](#)——它会降低覆盖度——而不是变成假，也不是变成未知。

四、评测：程序这一臂，严格的代价落在写入者身上

An LLM agent is given the recording task cold — five actions, no sight of Σ , the gate's verbatim refusals as the only feedback channel — and gets one revision. The original run went 2 accepted / 3 refused, then 5 of 5.

一个语言模型程序冷启动接到录入任务——五个动作，看不到治理规范，门给出的原话拒绝理由是唯一的反馈渠道——并且有一次修改机会。最初那次运行的结果是 2 条被接受、3 条被拒绝，修改之后 5 条全过。

We repeated the protocol across four Claude models, three trials each, with the scenario arranged so one refusal is not fixable by editing the record: household h3 belongs to a district the writer's authority does not reach.

我们在四个 Claude 模型上各做三次试验重复了这套流程，并且把场景设计成有一条拒绝[无法通过修改记录来解决](#)：h3 这户人家属于一个写入者的权限到不了的辖区。

Everything Σ can name converges: all label and vocabulary refusals were fixed in one revision in 12 of 12 trials, no trial invented a predicate, and the cheapest model gained the most — haiku

went 2/5 to full acceptance on its weakest attempts, which is where a gate that explains its refusals pays best.

凡是治理规范能指名的，都收敛了：所有标签类和词汇类的拒绝，在 12 次试验中的 12 次都在一轮修改内被修好，没有任何一次试验去发明一个谓词；而且最便宜的模型收益最大——haiku 在它最弱的那几次尝试上从 2/5 走到全部通过，而这正是「一道会解释自己为什么拒绝的门」价值最高的地方。

The authority refusal sorts writers by disposition, not capability: one trial made the correct move; all three opus trials declined every workaround as falsification and wrote their caveats into the record itself — refusing to trade truth for acceptance; five trials tried to park the record in a graph the writer does hold, and the gate caught every one.

那个权限拒绝把写入者按品性而不是按能力分开了：有一次试验做了正确的动作；三次 opus 试验把所有绕过方案都当成造假而拒绝，并把自己的顾虑写进记录本身——**不肯拿真实性去换通过率**；有五次试验想把记录停在写入者确实持有的那个图里，而门每一次都抓住了。

The gate is exactly as good as Σ : the one false record that landed passed because no policy states which district a household belongs to — the residual risk under refusal-driven convergence concentrates precisely in what Σ leaves unsaid, which is an argument for authoring coverage, not against the gate.

门的好坏正好等于治理规范的好坏：唯一一条落地的假记录之所以通过，是因为没有任何策略规定一户人家属于哪个区——在靠拒绝驱动收敛的机制下，剩余风险精确地集中在治理规范没说的地方，而这是「**应该把覆盖面写全**」的理由，不是「门没用」的理由。

SARC-DQ reports the ungated complement of the same design: agents handed defective evidence convert it into costly actions at a rate flat across four model tiers spanning $\sim 15\times$ in price — capability does not buy skepticism — so the convergence observed here is the gate's doing, not the models'.

同一设计的无门对照由 SARC-DQ 报告：把有缺陷的证据交给程序，它们把缺陷转化成昂贵动作的比率，在价格相差约 15 倍的四个模型档次上是平的——**能力买不到怀疑精神**——所以这里观察到的收敛是门的功劳，不是模型的功劳。

五、评测：真实轨迹与外部基准

Replaying the recorded Yupana trace through the audit yields $T \neq \Sigma$: 2 violations, 6 incompleteness findings over 5 records. The violations are real and actionable — the guard enforced a locally-configured blast-radius rule that is not an authored policy in Σ , and the finding's remediation is the thesis in one line: author it in the store so it can be audited, or stop enforcing it.

把录下来的 Yupana 真实轨迹放进审计重放，结果是「轨迹不满足规范」：5 条记录上有 **2 项违反、6 项不完备**。那两项违反是真实且可行动的——守卫强制执行了一条本地配置的影响半径规则，而那条规则**并不是治理规范里写下的策略**。这项发现的补救办法就是全篇论点的一句话版本：**要么把它写进库里让它可被审计，要么停止执行它。**

DEMM-Bench asks the converse: not whether decisions were correct, but whether the records a runtime emits suffice to reconstruct eight decision-level properties. Its lead diagnostic is Overclaim Rate — declaring a decision's evidence "sufficient" while a required property is not reconstructable — which operationalises the container fallacy: inferring audit sufficiency from the presence of a trace, ledger, or schema.

DEMM-Bench 问的是反面的问题：不是决定**对不对**，而是一个运行时发出的记录**够不够**重建出八个决策级属性。它的主诊断指标是**过度声明率**——在某个必需属性无法被重建的情况下，仍然宣称这个决定的证据「充分」。这个指标把**容器谬误**操作化了：从「有一条轨迹、有一本台账、有一个模式」推断出「审计证据充分」。

First, the container fallacy reproduces on Quipu's evidence at its ceiling: trace-, ledger-, and schema-presence baselines overclaim on 87.5% of cases, precisely because Quipu always emits all three planes — content-level degradation leaves every container present, so presence carries no information at all.

第一个发现：容器谬误在 Quipu 的证据上以**最高程度**重现——「有轨迹／有台账／有模式」这三个基线在 87.5% 的案例上过度声明，**恰恰是因为 Quipu 永远会把三个平面都发出来**。内容层面的劣化让每个容器都还在，所以「容器在不在」这件事根本不携带任何信息。

Third, the property-level reading reconstructs 512 of 512 property cells (mean PSA 1.0, zero overclaim, zero underclaim).

第三个发现：按属性逐格去读，512 个属性格全部重建成功（平均属性充分性 1.0，零过度声明，零声明不足）。

六、结论

The conventional knowledge graph was designed for a writer who no longer writes it alone.

传统的知识图谱，是为一个如今已不再独自写它的写入者而设计的。

The measured story is consistent: strictness is affordable when its cost falls on agents who can read a refusal and retry; refusals are worth recording as signed facts precisely because they are the events audit needs; composition can be made safe without being made silent; and a store whose rules are as historical as its data can replay not just what it knew but what it required.

测出来的结论是一致的：当严格的代价落在「能读懂一次拒绝并重试」的程序身上时，严格是负担得起的；拒绝值得作为签名事实记录下来，恰恰因为它们正是审计需要的那些事件；组合可以做到安全而不必做成静默；而一个「规则和数据一样具有历史性」的存储，能重放的不只是它当时知道什么，还有它当时要求什么。

Three boundaries are stated rather than hidden: no comparative query-performance claims; labels are not access control (a floor refuses a query, it does not hide rows); and denials replay as attestation checks, not re-derivations, because refusal-by-rollback deliberately keeps the verdict and discards the attempt.

三条边界是明说而不是藏起来的：不做查询性能的横向比较；[标签不是访问控制](#)（一个下限拒绝一次查询，它不隐藏任何行）；以及被拒绝的写入在重放时是核对签名证明，不是重新推导，因为「拒绝即回滚」这个设计故意保留了裁定、丢弃了那次尝试。

参考链接

- [Quipu 开发仓库](#) — 论文对应的可嵌入存储实现，单文件
- [归档产物 DOI 10.5281/zenodo.21878429](#) — 论文里每一个数字背后的源码与 benchmark/census 产物 (v0.3.20)
- [SARC 治理框架](#) — 把监管义务编译成运行时约束的框架，Quipu 的审计与它对照；同作者的 SARC-DQ 给出了「能力买不到怀疑精神」那条对照证据
- [camayoc](#) — 同作者在做的能力问题（competency question）测试套件，论文说它将替代目前那个脚本化场景作为质量判据